

9.第九讲：第一个项目--FishCake EventFi 项目实战

一.项目概况

1. 主要功能

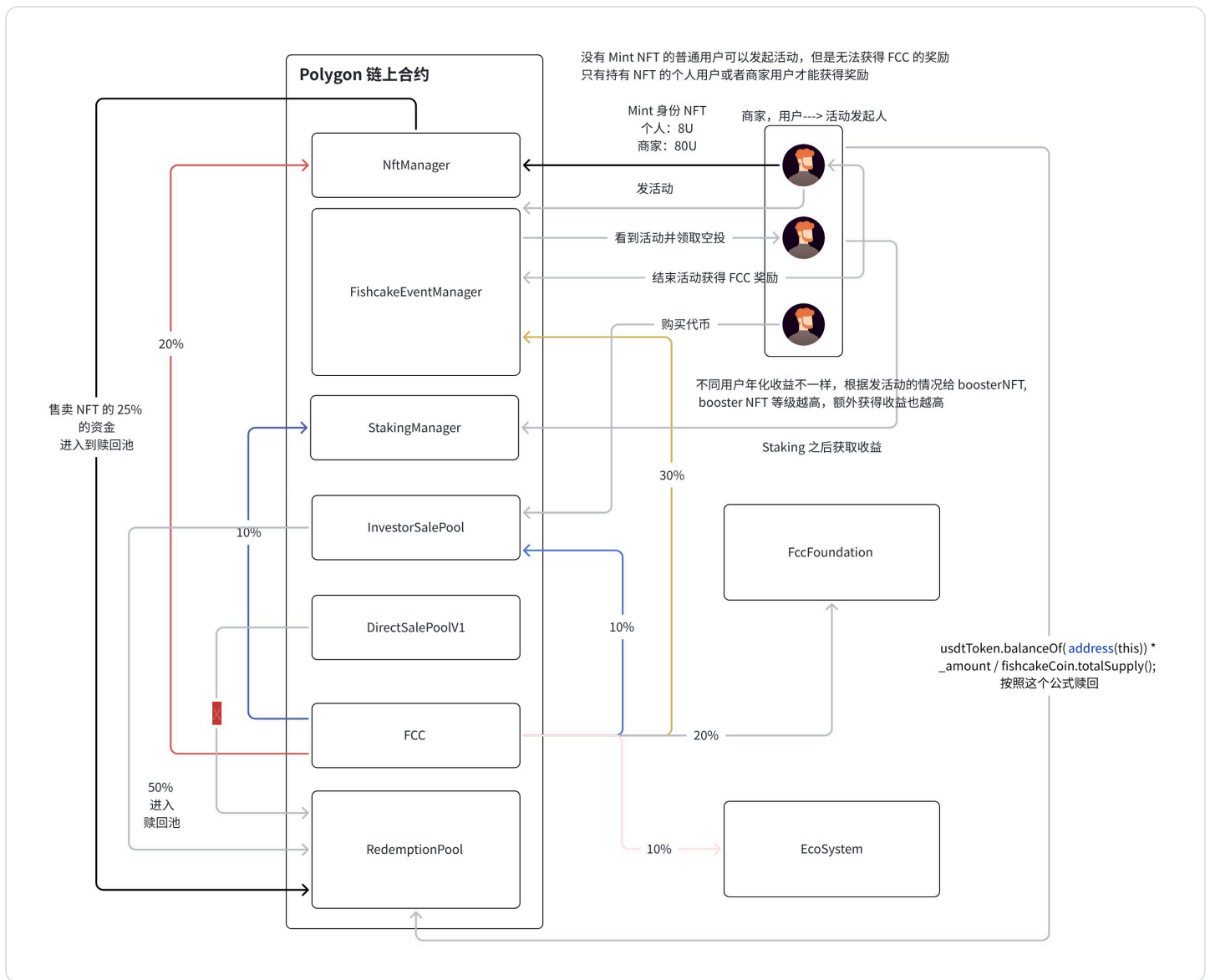
- NFT 会员
- 活动即挖矿
- 质押年化收益
- 钱包（不讲）
- 代币经济模型

2. 项目链接

- 官方链接
 - <https://fishcake.io/>
 - <https://fishcake.org/>
 - <https://www.fishcake.org/join> APP 下载
- 代码
 - 后端代码: <https://github.com/FishcakeLab/fishcake-service>
 - 合约代码: <https://github.com/FishcakeLab/fishcake-contracts>
 - APP: <https://github.com/FishcakeLab/FishCake>
 - H5: <https://github.com/FishcakeLab/fishcake-web3>

二.项目简介

FishCake 是一个 NFT 会员，集活动挖矿，代币经济模型，质押收益和钱包为一体化的一個项目



1. 系统角色

- 普通用户：未持有身份 NFT，可以发起和参与活动，但不能获得 FCC 活动奖励。
- 个人用户：支付 8U Mint 个人身份 NFT。
- 商家用户：支付 80U Mint 商家身份 NFT。
- 活动发起人：可以是普通用户、个人用户或商家用户。
- NFT 持有人：具备领取 FCC 活动奖励以及 Booster NFT 的资格。

2. FCC 初始分配

分配模块	比例	
FishcakeEventManager	30%	活动奖励、空投激励
NftManager	20%	NFT 相关奖励
StakingManager	10%	质押收益
InvestorSalePool	10%	投资人销售池
FccFoundation	20%	基金会储备
EcoSystem	10%	生态建设
合计	100%	—

3. 身份 NFT 业务流程

- 用户连接钱包并选择身份类型。
- 个人用户支付 8U，商家用户支付 80U。
- 用户通过 `NftManager` Mint 对应身份 NFT。
- `NftManager` 记录用户身份和 NFT 等级。
- NFT 销售收入的 25% 进入 `RedemptionPool` 赎回池。
- 用户获得活动奖励资格。

业务规则：

- 没有 NFT 的用户也可以发起活动。
- 没有 NFT 的用户不能获得 FCC 活动奖励。
- 只有持有个人 NFT 或商家 NFT 的用户才能获得 FCC 奖励。

4. 活动业务流程

- 用户或商家成为活动发起人。
- 发起人通过 `FishcakeEventManager` 创建活动。
- 活动内容、开始时间、结束时间、空投规则和奖励规则被记录在链上。
- 其他用户查看活动并参与。
- 符合条件的参与者领取活动空投。
- 活动结束后，由发起人或系统执行活动结算。
- `FishcakeEventManager` 检查用户是否持有有效身份 NFT。
- 符合条件的用户获得 FCC 奖励。
- 系统根据用户参与活动的情况发放 Booster NFT。

其中，活动奖励主要来自分配给 `FishcakeEventManager` 的 30% FCC。

5. Booster NFT 与质押流程

- 用户通过参与活动获得 Booster NFT。
- 系统根据活动次数、贡献度或完成情况确定 Booster NFT 等级。
- 用户将 FCC 存入 `StakingManager` 进行质押。
- `StakingManager` 读取用户的 Booster NFT 等级。
- 根据不同等级计算对应的收益权重。
- Booster NFT 等级越高，收益系数越高。
- 用户按照规则领取 FCC 质押收益。

质押奖励主要来自分配给 `StakingManager` 的 10% FCC。

6. FCC 购买流程

系统包含两类销售池：

- `InvestorSalePool`：面向投资人或特定用户销售，获得总量的 10% FCC。
- `DirectSalePoolV1`：面向普通用户直接销售 FCC。

基本流程：

- 用户选择购买 FCC。
- 用户支付 USDT。
- 对应销售池按照价格计算 FCC 数量。
- 销售池向用户发放 FCC。
- 图中标注销售资金的 50% 进入 `RedemptionPool`。

图中的“购买代币”线路经过 `FishcakeEventManager`，建议实际实现时统一由销售池负责收款和发币，避免活动合约同时承担代币销售职责。

7. FCC 赎回流程

`RedemptionPool` 的资金来源包括：

- NFT 销售收入的 25%。
- FCC 销售收入的 50%。
- 其他后续注入的赎回储备资金。

用户赎回时，按照图片中的公式计算：

赎回USDT数量=RedemptionPool的USDT余额×用户赎回FCC数量FCC总供应量
赎回USDT数量=
RedemptionPool的USDT余额 \times \frac{\text{用户赎回FCC数量}}{\text{FCC总供应量}}
赎回USDT数量
=RedemptionPool的USDT余额×FCC总供应量用户赎回FCC数量

对应代码逻辑：

代码块

```
1    usdtToken.balanceOf(address(this))
2        * _amount
3        / fishcakeCoin.totalSupply();
```

赎回流程：

- 用户提交需要赎回的 FCC 数量。
- 系统检查用户余额和授权额度。
- 计算用户可以获得的 USDT。
- 将用户提交的 FCC 销毁或转入回收地址。
- `RedemptionPool` 向用户支付 USDT。
- 更新赎回池余额和相关记录。

三.项目的架构设计

1. 合约层面

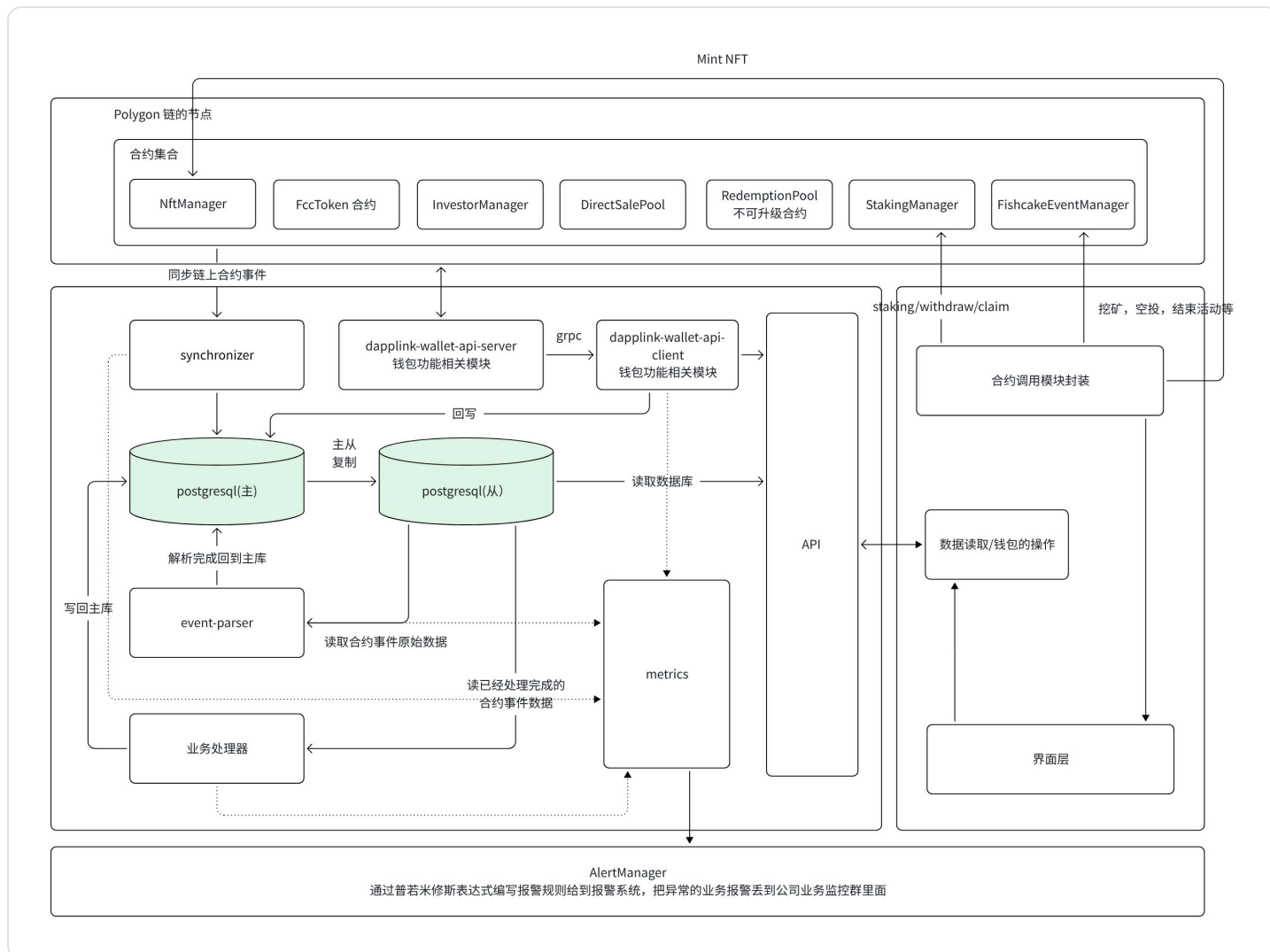
- Interface, Storage 和 logic 进行分离，继承和注意 slot 的处理
- 能做为常量的千万不要是变量，能 immutable 的一定要修饰成 immutable
- 注意权限的控制，权限分离，日常操作，资金操作，权限管理一定要分离
- 合约是时间锁，代币锁
- 合约升级模式----UUPS 或者透明代理都可以，这个项目用的是 UUPS
- 注意常见的攻击手段
- 那些合约设计成可升级的，那些合约设计成不可升级的需要严格注意
-

2. Go 服务层面

- 架构清晰明了
- 表结构设计，索引等等
- 注意处理链上回滚

- 主从数据库处理，多数据库表处理
- Metrics 指标明确
-

3. 项目整体架构设计

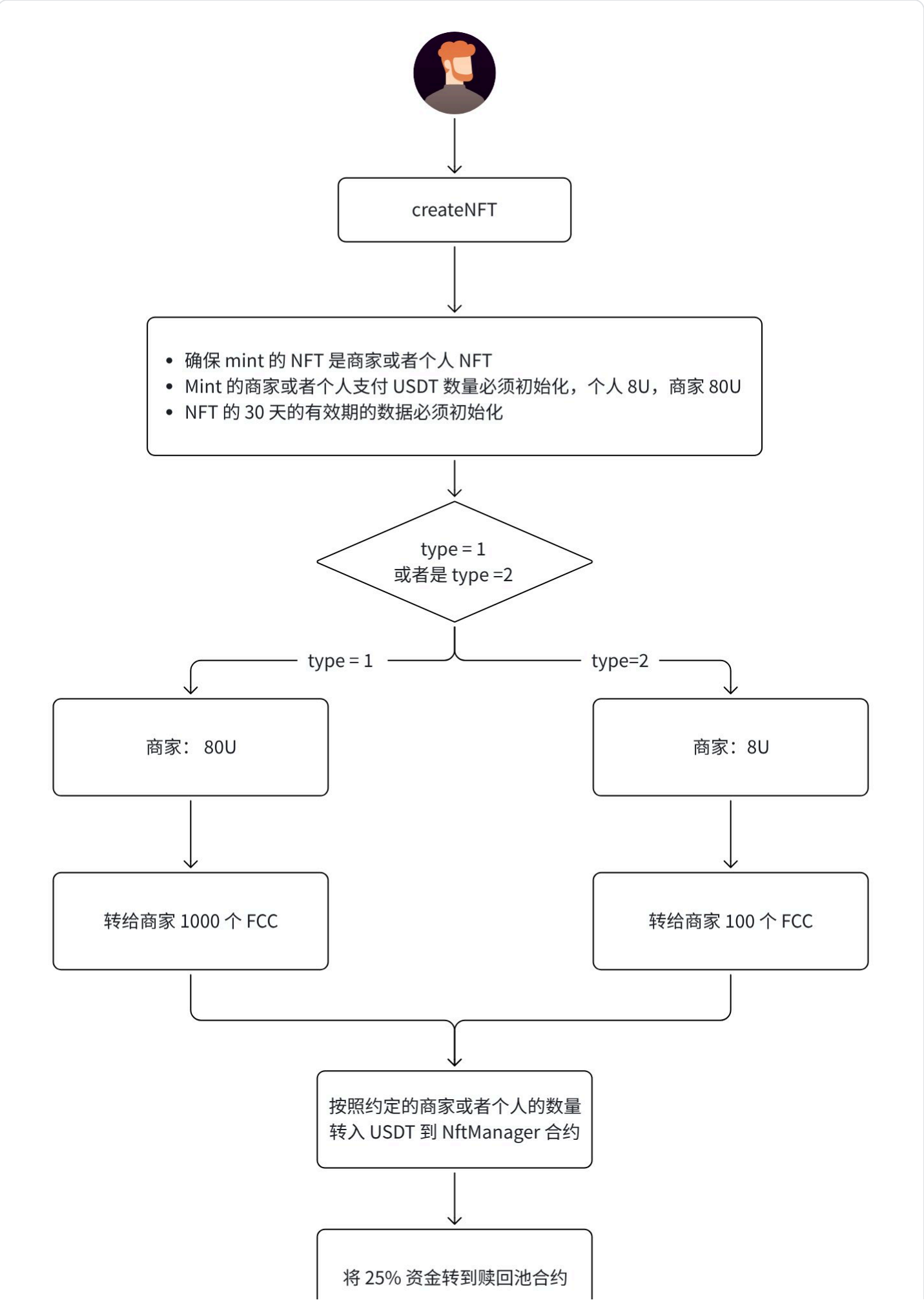


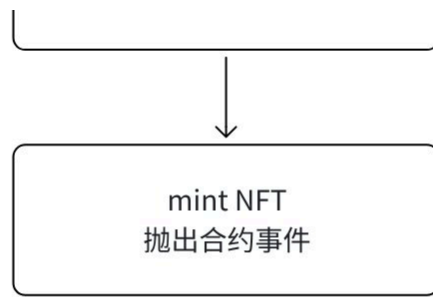
四. 合约代码讲解

1. NFT 合约 (NftManger)

- createNFT---> Mint 商家和个人 NFT 的方法
 - Mint 个人 NFT 花费 8U, mint NFT 完成之后获得是 100 FCC
 - Mint 上海 NFT 花费是 80, mint NFT 完成之后获得是 1000 FCC
 - Mint 的 USDT 有 25% 进入赎回池, 75% 留在当前的 NFT 合约
 - Mint NFT 的有效期是 30 天

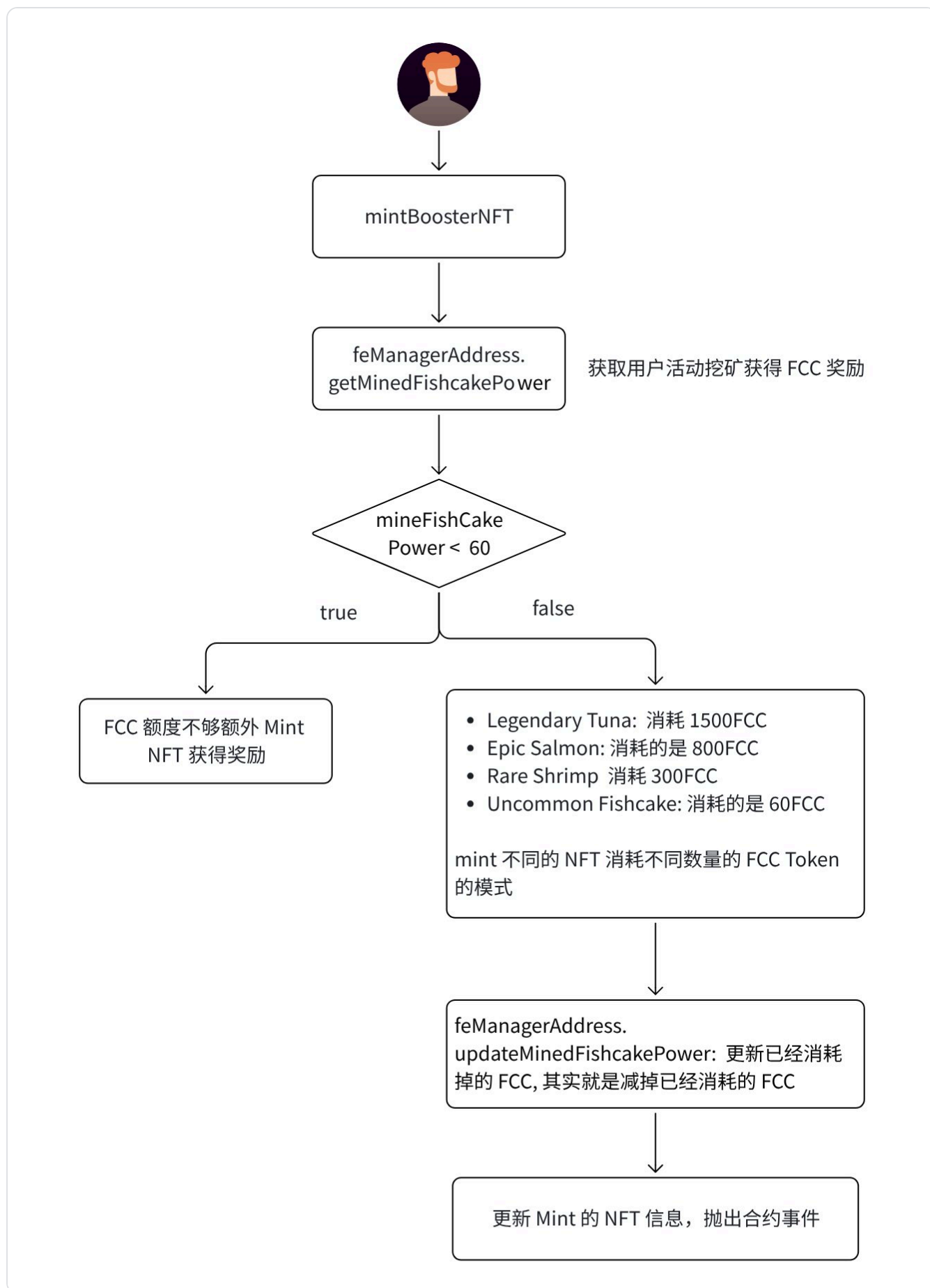
• Mint 的 NFT 的流程图





- WithdrawToken 和 WithdrawNativeToken
 - WithdrawToken: 将合约里面的 ERC20 Token 提到指定的地址里面
 - WithdrawNativeToken: 将合约里面的 Native Token 提到指定的地址里面
- mintBoosterNFT
 - BoosterNFT 质押时候持有 boosterNFT 参与质押的话，将会获得额外，这里讲 staking 代码的时候详细讲解
 - 用户传入用 mint NFT 的类别

等级	NFT 名称	基础 APR 加成 (2025 后使用减半)	发放（根据leaderboard）
Tier 1	Legendary Tuna	+20% APR	当月ming = 1500FCC
Tier 2	Epic Salmon	+15% APR	当月mining ≥ 800 FCC
Tier 3	Rare Shrimp	+9% APR	当月mining =300FCC
Tier 4	Uncommon Fishcake	+5% APR	当月mining =60FCC



2. FishcakeEventManager

- 发起活动: `activityAdd`



activityAdd

- 空投类别只能是 1 和 2，1 是平均空投，2 是随机空投
- 最大的空投数量一定要大于最大的空投数量
- 最大的空投数量必须大于 0
- 活动的截止时间必须大于当前去区块时间戳小于最大截止时间
- 总的空投的资金数量要等于最大空投数量乘以空投份数
- 最大空投数量要大于 1 代币
- 空投的份数最大是 100 份
- 空投的 Token 只能是 FCC 或者 USDT

将空投的资金转入到合约里面

transferFrom

活动和活动信息创建
创建完成丢到 List 里面

抛出 ActivityAdd 合约
事件

- 活动空投：drop



- 结束活动: activityFinish



activityFinish



活动 Id 有效性: 大于 0 小于最大 ID, 最大 ID 就是活动的数组长度

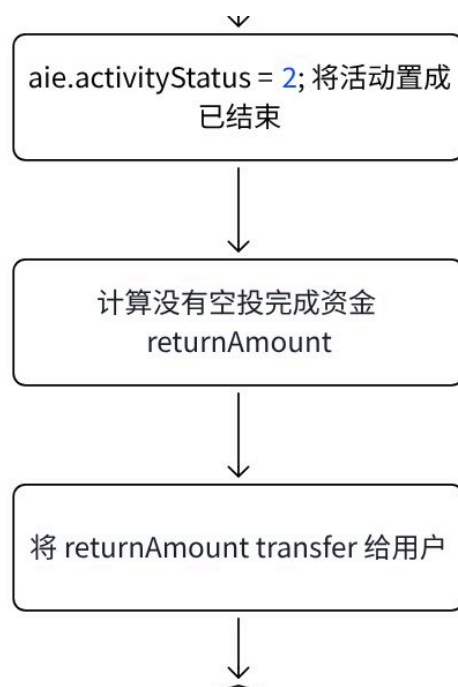


提取活动信息



- 活动的结束人是活动的 Owner
 - 判断活动是否已经结束





3. StakinManager

3.1 FCC Staking 收益和 BoosterNFT 收益

3.1.1 固定 APR 结构（基础收益）

锁仓天数	APR (年化)	计算规则	实际收益%
30	3%	$APR / 365 * 30$	0.25%
60	6%	$APR / 365 * 60$	0.99%
90	9%	$APR / 365 * 90$	2.22%
180	15%	$APR / 365 * 180$	7.40%

3.1.2 FCC Staking Booster

Booster NFT Mint 流程：官方空投用途（非公开Mint）

Fishcake 的 Booster NFT 是不可自由铸造的功能性 NFT，只能由项目方根据 Leaderboard 或其他激励策略 按需 Mint 给用户，确保公平与控制通胀。

等级	NFT 名称	基础 APR 加成 (2025 后使用减半)	发放（根据leaderboard）
Tier 1	Legendary Tuna	+20% APR	当月ming = 1500FCC
Tier 2	Epic Salmon	+15% APR	当月mining $\geq$ 800 FCC
Tier 3	Rare Shrimp	+9% APR	当月mining =300FCC
Tier 4	Uncommon Fishcake	+5% APR	当月mining =60FCC

3.2 NFT Booster 使用原理

- 原则：
 - Booster NFT 可一次性绑定到一笔质押；Booster NFT 属于额外的质押收益，如果没有 Booster NFT，只能获得固定的收益
 - 使用后该 Booster NFT 标记为“已使用”，不能重复。
- 例效果（以 90 天为例）：
 - 基础 APR: 9%
 - 使用 Epic Booster（+15% 年化）
 - 实际收益 %: $\{(9\%+15\%)/365\} * 90 = 5.92\%$
- USDT 空投奖励（代码里面看不出来）
 - 根据 leaderboard 的榜单前 n 质押，随机抽取 5-10 名幸运瓜分 1000U，按照质押量

3.3 代码逻辑梳理

- depositIntoStaking



depositIntoStaking

- 质押金额大于最小质押金额
- 把质押的资金转入到 staking 的合约

getStakingPeriodAndApr(stakingType)
按照用户传入的 stakingType 获取质押周期

获取 NFT APR 年化收益绑定到质押的交易

nftManagerAddress.inActiveMinerBoosterNft
将 NFT 置成已使用状态

生成 stakingQueued 交易信息丢到
stakingQueued 里面

更新总的质押抛出合约事件

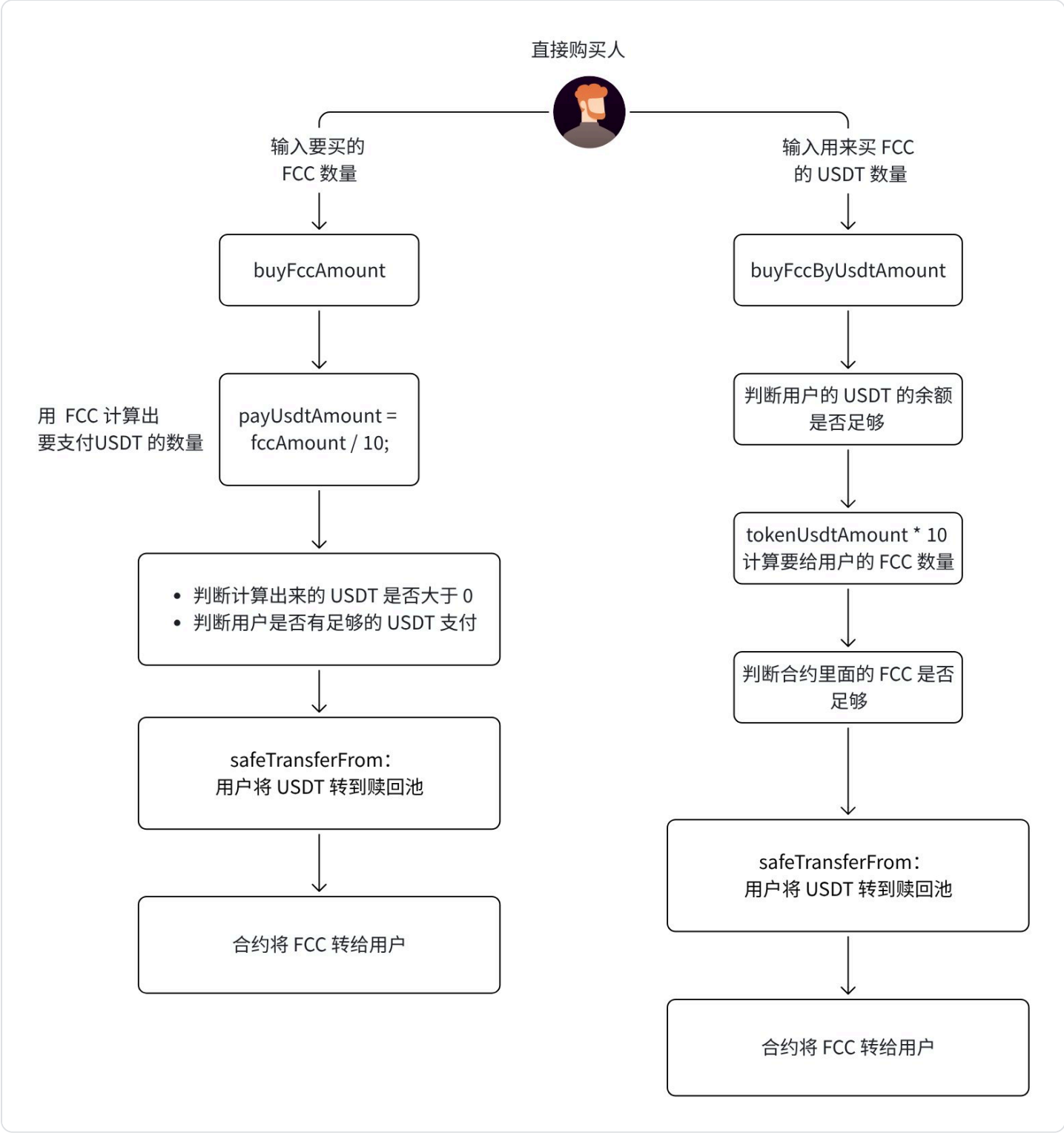
- withdrawFromStakingWithAprIncome



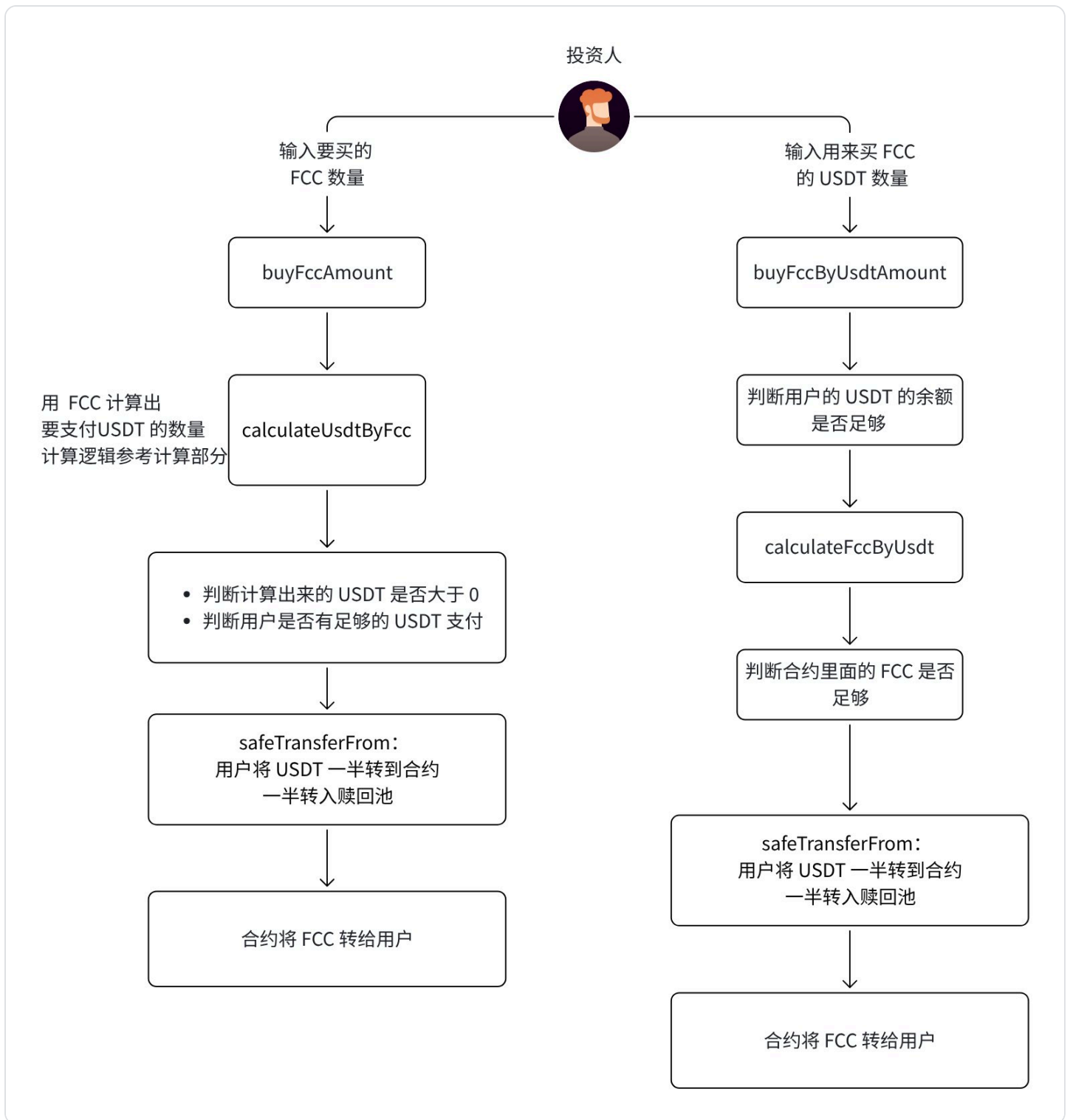
withdrawFromStakingWithAprl
ncome

4. 销售合约

• DirectSalePoolV1



• InvestorSalePool



计算的逻辑

- 按照 FCC 的数量计算
 - $x \geq 5000000$ 万 FCC; $1 \text{ FCC} = 0.06 \text{ USDT}$
 - $250_000 \leq x < 50$ 万 FCC; $1 \text{ FCC} = 0.07 \text{ USDT}$
 - $100_000 \leq x < 250_000$; $1 \text{ FCC} = 0.08 \text{ USDT}$
 - $16_666 \leq x < 100_000$; $1 \text{ FCC} = 0.09 \text{ USDT}$
 - $< 16_666$; $t \text{ FCC} = 0.1 \text{ USDT}$
- 按照 USDT 的数量计算

- $X \geq 100_000$; 1 FCC = 0.06 USDT
- $10_000 \leq X < 100_000$; 1 FCC = 0.07 USDT
- $5_000 \leq X < 10_000$; 1 FCC = 0.08 USDT
- $X < 1_000$; 1 FCC = 0.09 USDT

5. FCC 代币合约

- 发币的逻辑

代码块

```
1  function poolAllocate() external onlyOwner {
2      _beforeAllocation();
3      _mint(fcPool.miningPool, (MaxTotalSupply * 3) / 10); // 30% of total supply
4      _mint(fcPool.directSalePool, (MaxTotalSupply * 2) / 10); // 20% of total
    supply
5      _mint(fcPool.investorSalePool, MaxTotalSupply / 10); // 10% of total supply
6      _mint(fcPool.nftSalesRewardsPool, (MaxTotalSupply * 2) / 10); // 20% of
    total supply
7      _mint(fcPool.ecosystemPool, MaxTotalSupply / 10); // 10% of total supply
8      _mint(fcPool.foundationPool, MaxTotalSupply / 10); // 10% of total supply
9      isAllocation = true;
10 }
```

- 活动挖矿的：30%
- directSalePool 直接销售：20%
- investorSalePool 投资者销售：10%
- nftSalesRewardsPool：NFT 销售 20%
- ecosystemPool 生态池：10%
- foundationPool 基金会 10%

6. 赎回池合约

赎回池资金锁定时间：uint256 public immutable unlockTime = block.timestamp + 1095 days; 其实是 3 年

用户赎回 USDT 的流程

赎回 USDT 的用户



claim

- 赎回池的合约是否已经解锁
- 用户赎回的 FCC 余额是否足够

calculateUsdt

根据用户输入的 FCC 数量计算出能赎回的 USDT 的数量

- 判断计算出来的 USDT 是否大于0
- 判断赎回池的资金是否足够支付给用户

burn

销毁用户转入的 FCC

safeTransfer

将 USDT 转给用户

FCC 转换成 USDT 的计算逻辑讲解：

代码块

```
1 usdtToken.balanceOf(address(this)) * _amount / fishcakeCoin.totalSupply();
```

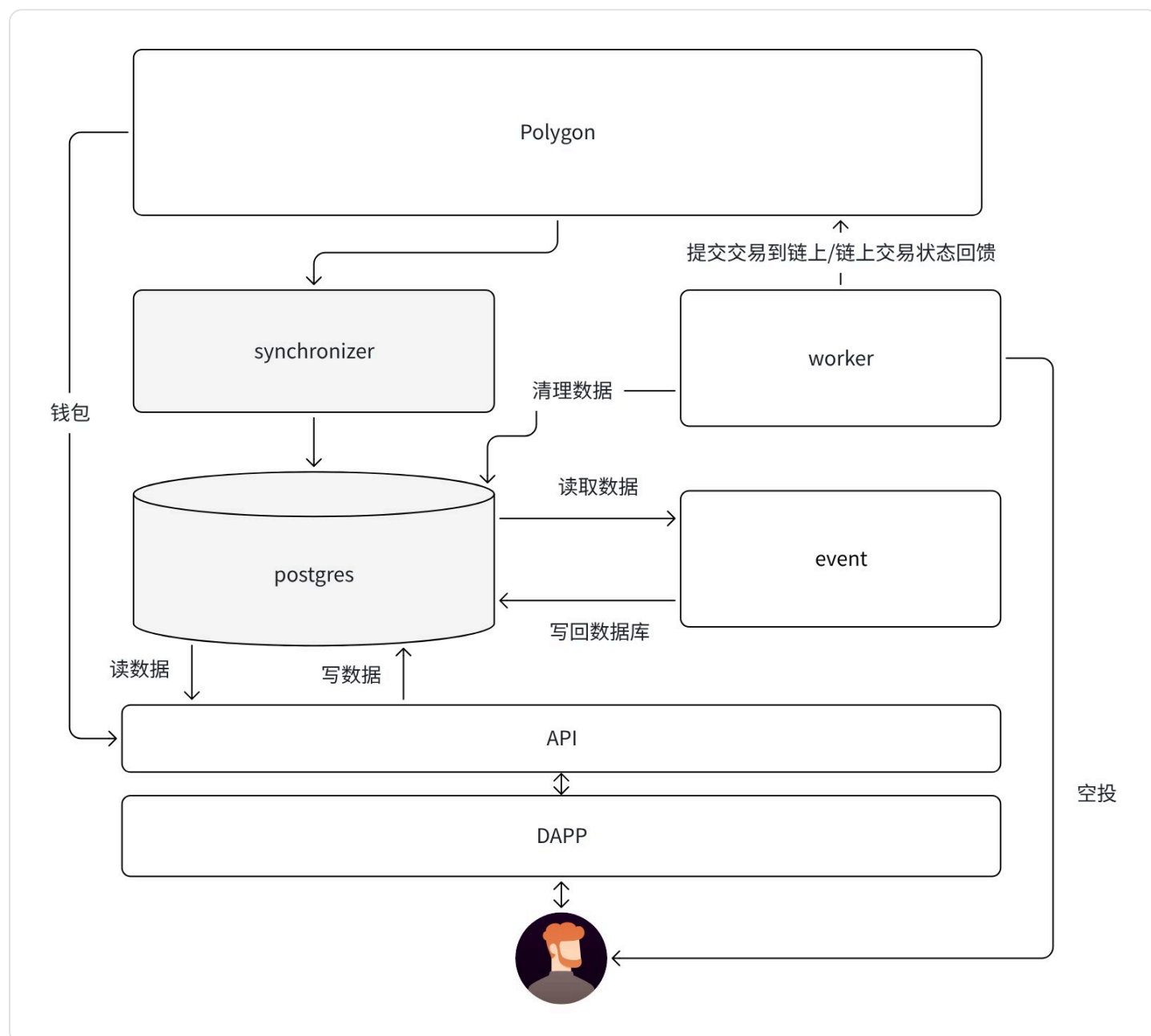
例子：

- 赎回池有 10000000 USDT

- 总发行量是：10000000000 FCC
- 用户 A 用 10000 FCC 赎回 USDT
 - $10000000 * 10000 / 100000000000 = 10 \text{ USDT}$
 - 动作完成之后
 - USDT: 9999990
 - FCC: 9999990000
- 用户 B 用 10000 FCC 赎回 USDT
 - $9999990 * 10000 / 9999990000 = 10 \text{ USDT}$

五. Go 代码的讲解

1. 代码架构

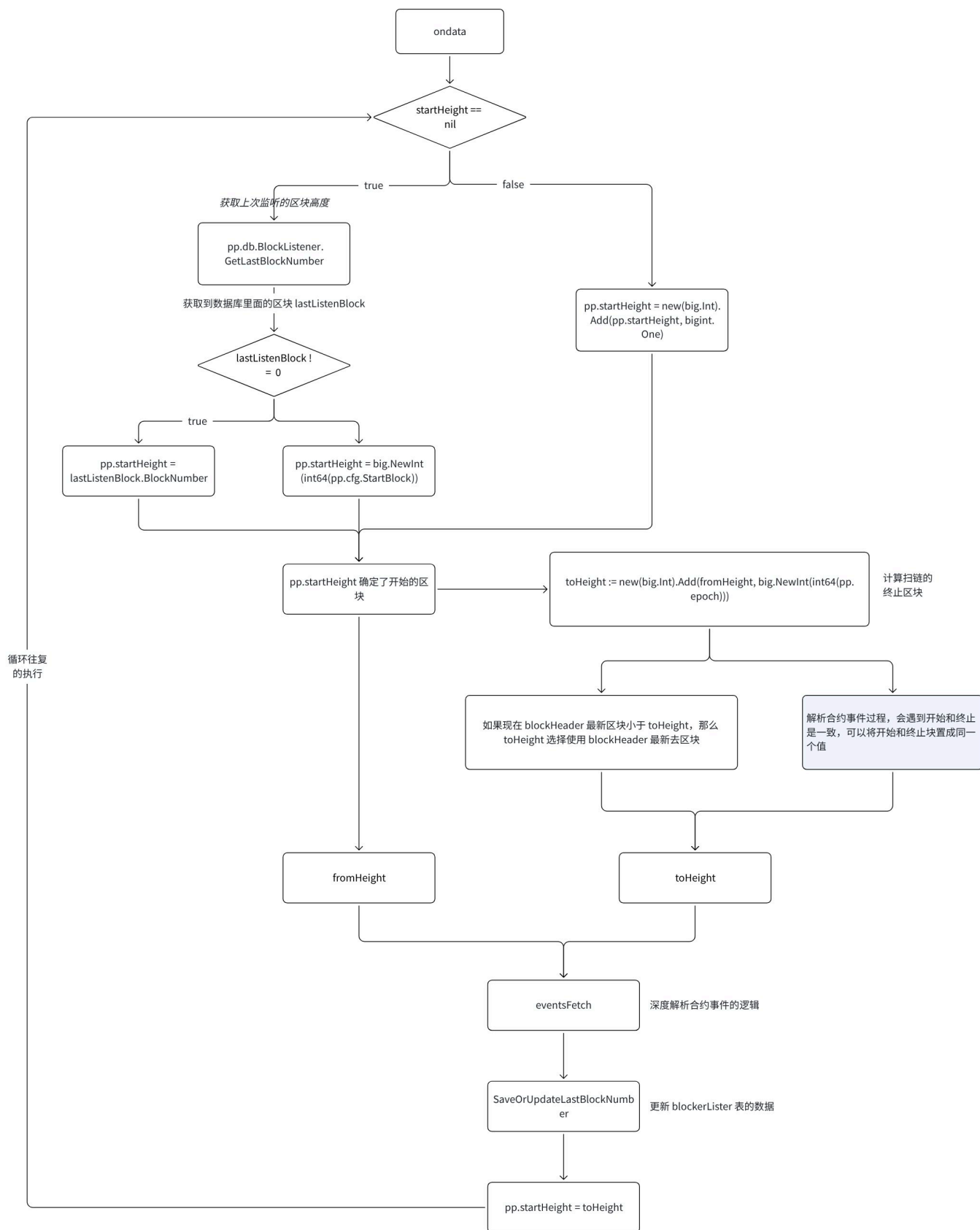


2. 合约事件监听

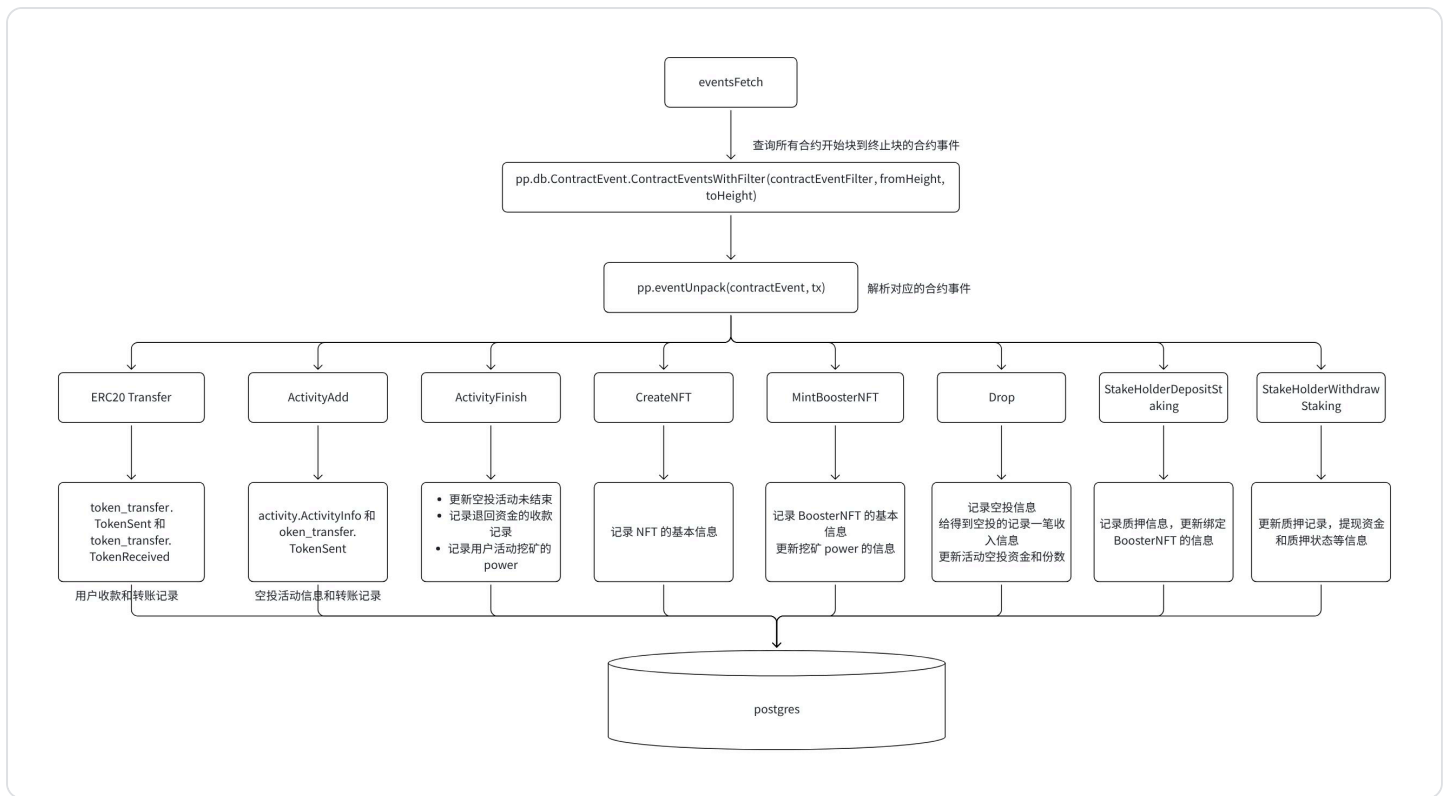
- 参考前面讲过的代码，和其他项目是类似的，这里不再赘述

3. 合约事件结合业务解析

- 解析合约事件总体业务逻辑



- 解析合约事件的详情



4. 业务进程讲解

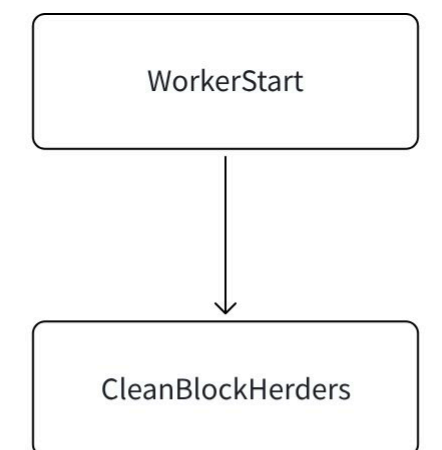
4.1 简介

Worker 一共有有 4 个人任务，分别承载区块数据的清理，系统空投和钱包创建空投的逻辑，交易的上链处理队列

- clean_data_worker: 清理 block_headers 的数据，清理之后只留下最新的 200000 条数据，这种处理方式有一定的问题，实际需要把数据进行分表处理，持久化历史数据。
- drop_worker 和 system_drop_worker: 第一次和活动合约交互的人会获得空投
- submit_tx_worker: 前端进行交易离线签名之后，发送后端 queue_tx 表，这个进程来将这些交易发送链上，并确保交易能够成功。

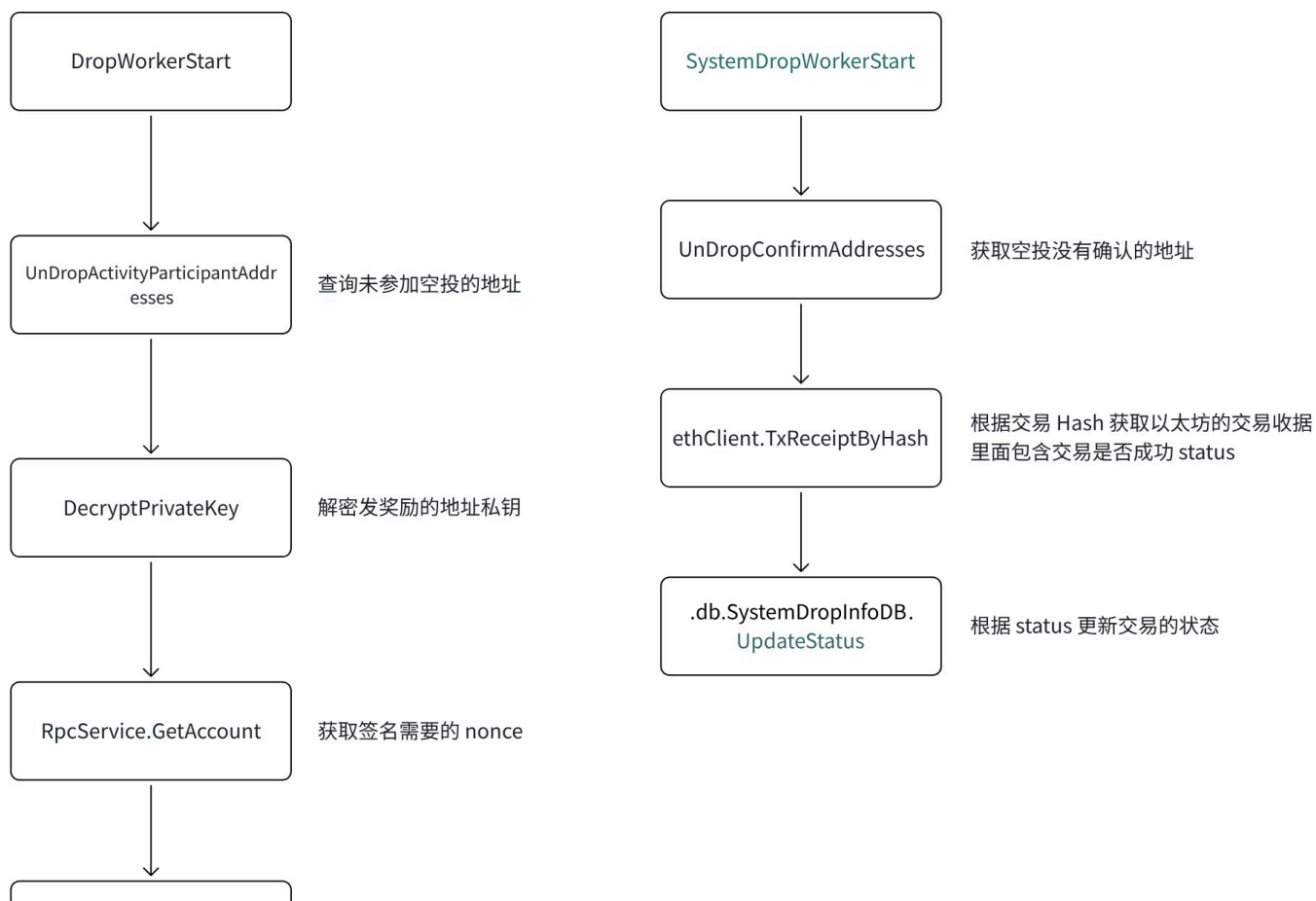
4.2 详细的业务流程

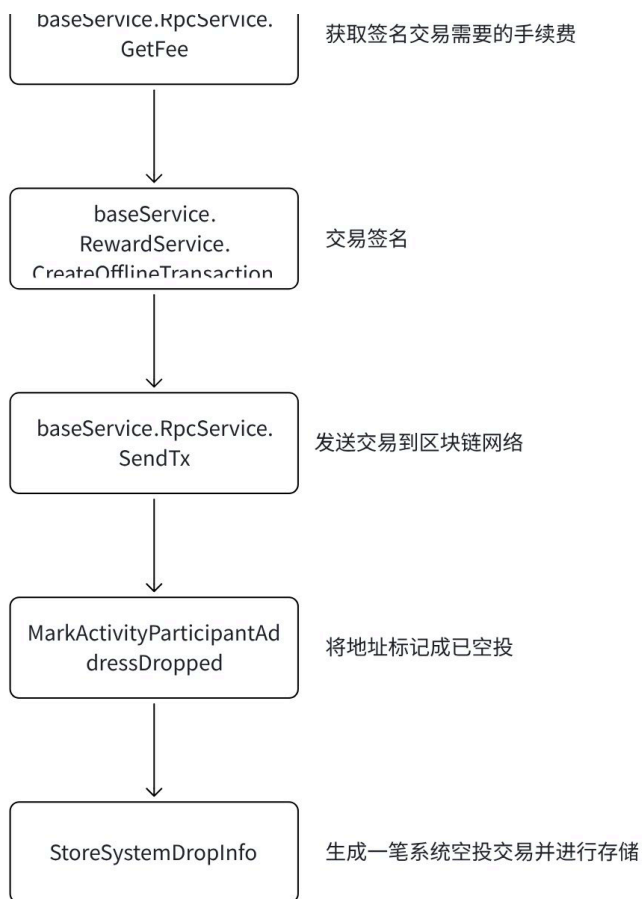
4.2.1 clean_data_worker



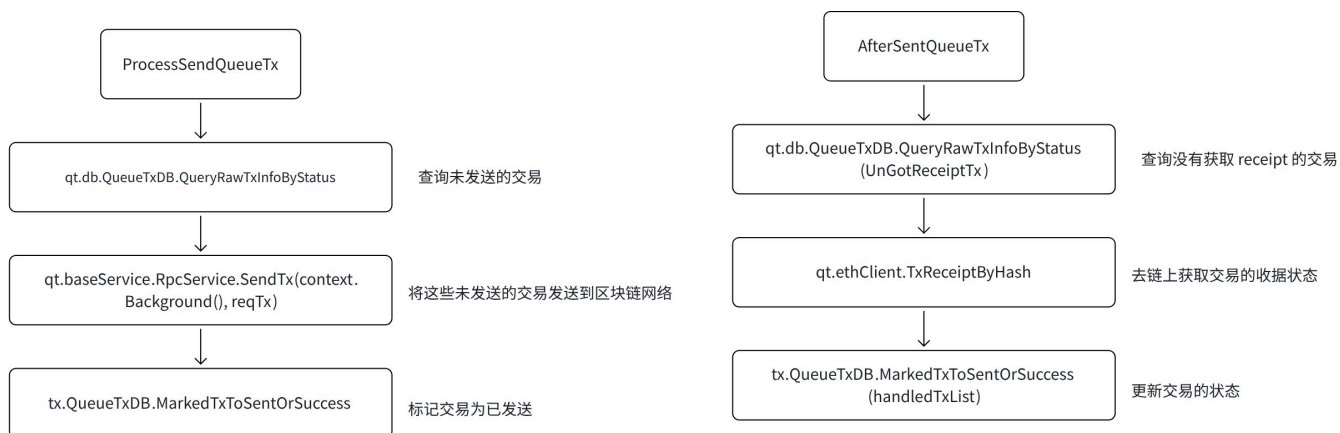
保留最新的 20000 块，其他的删除

4.2.2 drop_worker 和 system_drop_worker

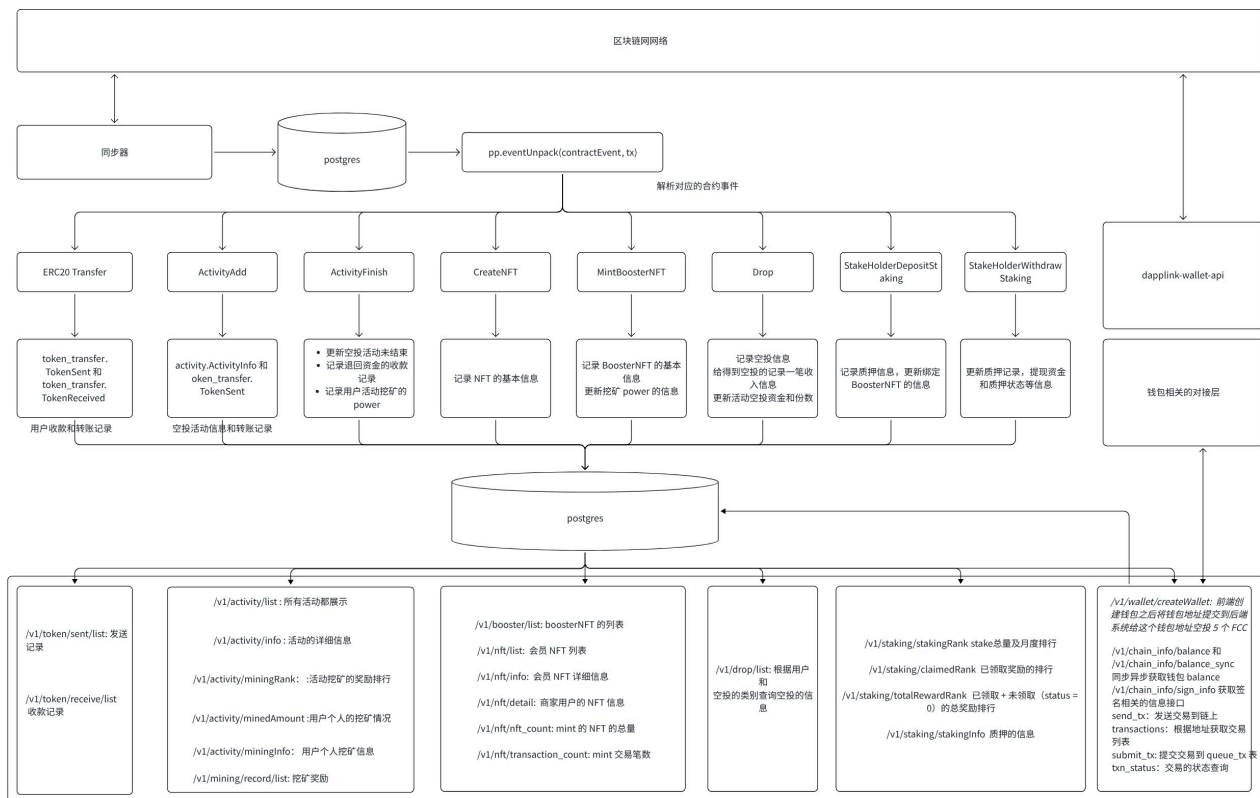




4.2.3 queue_tx



5. API 讲解



- **/v1/wallet/createWallet:** 这个 API 是存在问题，他不管代币的空投是否成功，只是空投了，其他都不管，需要改进成为后台任务，获取交易的收据树，若失败需要再次空投。

6. 代码总结

合约代码: <https://github.com/FishcakeLab/fishcake-contracts>

Go 服务代码: <https://github.com/FishcakeLab/fishcake-service>

代码经济模型: <https://github.com/FishcakeLab/Fishcake-Tokenomics>

Wallet 对接链上: <https://github.com/daplink-labs/wallet-chain-node>

Daplink-wallet-api: <https://github.com/daplink-external/daplink-wallet-api>

rpc 负载: <https://github.com/FishcakeLab/fishcake-rpc>

六. BNB 测试网运行测试

1. 合约的部署

- 地址准备

代码块

```
2 Successfully created new keypair.
3 Address:      0xd449306df27faB97ec0431F4558ab75966743983
4 Private key: 0xb857bcf6f3ee005a79eea257a4ab467a3488b5006fa4a9dbba7d54a9a41aafa5
```

- 去 github 上 clone 代码

代码块

```
1 git clone git@github.com:FishcakeLab/fishcake-contracts.git
2
3 forge build 自动安全依赖和构建
```

- 配置环境变量

代码块

```
1 export
  PRIVATE_KEY=0xb857bcf6f3ee005a79eea257a4ab467a3488b5006fa4a9dbba7d54a9a41aafa5
2 export BOOSTER_ADDRESS=0xd449306df27faB97ec0431F4558ab75966743983
```

- 部署合约

代码块

```
1 forge script ./script/DeployerScript.sol:DeployerScript --rpc-url https://bsc-
  testnet.bnbchain.org --broadcast
```

- 部署之后得到的地址

代码块

```
1 No files changed, compilation skipped
2 Script ran successfully.
3
4 == Logs ==
5 Deploying contracts with account: 0xd449306df27faB97ec0431F4558ab75966743983
6 Deployed Mock USDT: 0x32295C8a779cDA5F71E99Bb6605cE115511a3332
7 FishCakeCoin implementation: 0xfBf5F5dcDefEEB7955B1ED4F36338E66c6052a7f
8 FishCakeCoin proxy: 0xB0BBC44BAA860711F32a697Cde76014535BC4BA7
9 RedemptionPool: 0x95190a246f31bDF771B16F8c8CD7f9Ecce8d5D03
10 DirectSalePoolV1 implementation: 0xEf2737aD9A2C47d61a3523b8C1289DCcAb70513E
11 DirectSalePoolV1 proxy: 0x7DeFfa3488dE0DB1e990eb3bB1d20dC385fAea24
12 InvestorSalePool implementation: 0x4733F1ad837351D0A7B439061745471F6bc4Be3C
13 InvestorSalePool proxy: 0x841c152EA131eE5c8D3a2e799FC20AB69D0F17e3
```

```
14 NftManagerV5 implementation: 0x43887166f75C423fB2a7ADEcaE5Cfb85977F97E0
15 NftManagerV5 proxy: 0x2282F52bf0a13BfD1b0135209303a4A7b23bB0AA
16 FishcakeEventManagerV2 implementation:
0x5c2E2401929d590c55134d6b76a545011F0CEd11
17 FishcakeEventManagerV2 proxy: 0xA7f5734cC2e5c3D0F10783f41467499284706C85
18 StakingManager implementation: 0xCe8b00A6afF37cd812976bFa8cf1E98f15ba1Cdb
19 StakingManager proxy: 0xF818C5667520Be9CBBdB7831864F5286Ffbdb14d
```

FCC Token 的发行

- 设定接收发出来币种的地址

代码块

```
1 cast send 0xB0BBC44BAA860711F32a697Cde76014535BC4BA7
'setPoolAddress((address,address,address,address,address,address,address))'
'(0xF818C5667520Be9CBBdB7831864F5286Ffbdb14d,0x7DeFfa3488dE0DB1e990eb3bB1d20dC3
85fAea24,0x841c152EA131eE5c8D3a2e799FC20AB69D0F17e3,0x2282F52bf0a13BfD1b0135209
303a4A7b23bB0AA,0xA7f5734cC2e5c3D0F10783f41467499284706C85,0xd449306df27faB97ec
0431F4558ab75966743983,0x95190a246f31bDF771B16F8c8CD7f9Ecce8d5D03)' --rpc-url
'https://go.getblock.io/40740512324a466bb572c9f6455c274f' --private-key
"$PRIVATE_KEY"
```

- 发布

代码块

```
1 cast send 0xB0BBC44BAA860711F32a697Cde76014535BC4BA7 "poolAllocate()" --rpc-
url 'https://go.getblock.io/40740512324a466bb572c9f6455c274f' --private-key
"$PRIVATE_KEY"
```

Mint NFT

- Approve token

代码块

```
1 cast send --rpc-url "https://go.getblock.io/40740512324a466bb572c9f6455c274f"
--private-key "$PRIVATE_KEY" 0x32295C8a779cDA5F71E99Bb6605cE115511a3332
"approve(address,uint256)" 0x2282F52bf0a13BfD1b0135209303a4A7b23bB0AA
800000000000
```

- Mint NFT

代码块

```

1  cast send "$CONTRACT_ADDRESS" \
2    "createNFT(string,string,string,string,string,string,uint8)" \
3    "商家名称" \
4    "商家描述" \
5    "https://example.com/nft.png" \
6    "商家地址" \
7    "https://example.com" \
8    "https://x.com/example" \
9    1 \
10  --rpc-url "https://go.getblock.io/40740512324a466bb572c9f6455c274f" \
11  --private-key "$PRIVATE_KEY"

```

发活动

代码块

```

1  RPC_URL='https://go.getblock.io/40740512324a466bb572c9f6455c274f'
2  EVENT_MANAGER='0xA7f5734cC2e5c3D0F10783f41467499284706C85'
3  USDT='0x32295C8a779cDA5F71E99Bb6605cE115511a3332'
4  DEADLINE=$(( $(date +%s) + 600 ))
5  TOTAL_DROP_AMTS=10000000000
6
7  cast send "$USDT" 'approve(address,uint256)' "$EVENT_MANAGER"
8  "$TOTAL_DROP_AMTS" --rpc-url "$RPC_URL" --private-key "$PRIVATE_KEY"
9
10 cast send "$EVENT_MANAGER"
11 'activityAdd(string,string,string,uint256,uint256,uint8,uint256,uint256,uint256,
12 address)' 'FishCake深圳活动' '参与线下活动领取奖励' '22.543096,114.057865'
13 "$DEADLINE" 10000000000 1 100 0 100000000 "$USDT" --rpc-url "$RPC_URL" --private-
14 key "$PRIVATE_KEY"

```

2. 链下服务的启动

- 克隆代码

代码块

```

1  git clone git@github.com:FishcakeLab/fishcake-service.git
2
3  go mod tidy

```

- 编译代码

代码块

```
1 make
```

- 配置启动配置文件

代码块

```
1 migrations: "./migrations"
2
3 # Polygon RPC (用公共节点即可, 读区块够用)
4
5 polygon_rpc: "https://go.getblock.io/40740512324a466bb572c9f6455c274f"
6
7 rpc_url: "tw-wallet-chain-account:80"
8 polygon_chain_id: "137"
9
10 http_host: "127.0.0.1"
11 http_port: 8189
12
13 db_host: "127.0.0.1"
14 db_port: 5432
15 db_name: "fishcakes9"
16 db_user: "guoshijiang"
17 db_password: ""
18
19 metrics_host: "127.0.0.1"
20 metrics_port: 9090
21
22 # 同步起始区块
23
24 start_block: 124371300
25 event_start_block: 124371300
26
27 # 三个合约地址 (都用合法的格式即可)
28 contracts:
29     - "0x2282F52bf0a13BfD1b0135209303a4A7b23bB0AA"
30     - "0xA7f5734cC2e5c3D0F10783f41467499284706C85"
31     - "0xF818C5667520Be9CBdB7831864F5286Fbdb14d"
32
33 # 阿里云配置 (本地无调用逻辑时填假值防空指针)
34 ali_config:
35     region_id: "cn-hangzhou"
36     access_key: "FAKE_ACCESS_KEY"
37     access_secret: "FAKE_ACCESS_SECRET"
38
39 # 合约依赖映射 (都填上有效格式地址即可)
40 contract_info:
```

```

41     fcc_token: "0xB0BBC44BAA860711F32a697Cde76014535BC4BA7"
42     usdt_token: "0x32295C8a779cDA5F71E99Bb6605cE115511a3332"
43     redemption_pool: "0x95190a246f31bDF771B16F8c8CD7f9Ecce8d5D03"
44     direct_sale_pool: "0x7DeFfa3488dE0DB1e990eb3bB1d20dC385fAea24"
45     investor_sale_pool: "0x841c152EA131eE5c8D3a2e799FC20AB69D0F17e3"
46     nft_manager: "0x2282F52bf0a13BfD1b0135209303a4A7b23bB0AA"
47     fishcake_event_manager: "0xA7f5734cC2e5c3D0F10783f41467499284706C85"
48
49     # 加密字段防空指针: 随便填假值
50     encrypted_private_key:
51         "649ce5ddf798e74c6d226c96f19e9113eabf581ac28e830164f9f2b2b9b55a4a30467dff788529
52         ae8582998efa8e1753"
53
54     nonce: "97e81dfb6b69df261af1471a"
55     key_phrase: "fishcake-test-key-phrase"
56
57     # Token 映射
58     usdt: "0x32295C8a779cDA5F71E99Bb6605cE115511a3332"
59     fcc: "0xB0BBC44BAA860711F32a697Cde76014535BC4BA7"
60
61     # PolygonScan API (假的 key 也没关系, 主要是防止空字符串)
62     data_api_url: "https://api-testnet.bscscan.com/api?"
63     data_api_key: "TEST_KEY_FOR_LOCAL"

```

- 执行 migrate 生成数据库

代码块

```
1 ./fishcake migrate
```

- 启动扫链服务

代码块

```
1 ./fishcake index
```

- 启动 api 服务

代码块

```
1 ./fishcake api
```

- 测试 API


```

1 代码块 http://127.0.0.1:8189/v1/activity/list?pageSize=1&pageNum=1
2
3 - 返回值
4 {
5   "success": true,
6   "msg": "success",
7   "obj": {
8     "currentPage": 1,
9     "pageSize": 1,
10    "result": [
11      {
12        "id": "3b1d234bcf1e4977a1e64cd79355b871",
13        "activityId": 1,
14        "businessAccount": "0xd449306df27faB97ec0431F4558ab75966743983",
15        "businessName": "FishCake深圳活动",
16        "activityContent": "参与线下活动领取奖励",
17        "latitudeLongitude": "22.543096,114.057865",
18        "activityCreateTime": 1786416159,
19        "activityDeadline": 1786416745,
20        "dropType": 1,
21        "dropNumber": 100,
22        "minDropAmt": 0,
23        "maxDropAmt": 100000000,
24        "tokenContractAddr": "0x32295C8a779cDA5F71E99Bb6605cE115511a3332",
25        "activityStatus": 1,
26        "alreadyDropNumber": 0,
27        "basicDeadline": 0,
28        "proDeadline": 1789007404,
29        "returnAmount": 0,
30        "minedAmount": 0,
31        "isExpired": false
32      }
33    ],
34    "total": 1
35  },
36  "code": "0000"
37 }

```

- 启动钱包的功能一定链接到：<https://github.com/dapplink-labs/wallet-chain-account>

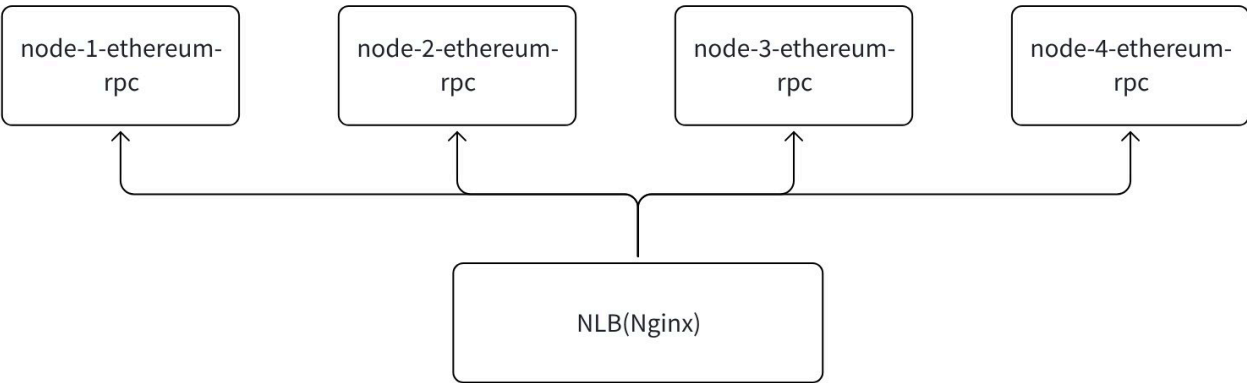
七.项目实施过程中会遇到的问题

1. 实际过程做回到 RPC 单节点问题，单节点会挂，一般会使用节点负载

RPC 节点在大一些公司，一般都是自建，自建之后做负载；在小一点的公司或者团队，用第三方的节点

1.1 自建节点

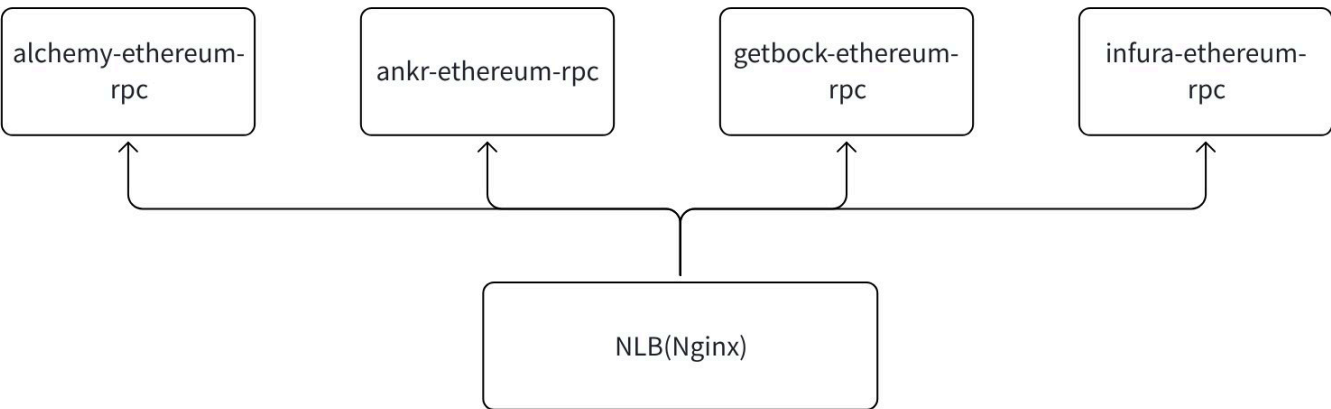
- 节点活性问题：每条链节点出块时间基本是确定，超过一定时间不出块这个节点认为是有问题的，可以负载均衡网络踢出这个节点，等正常了在纳入进来
- 最新区块：以负载均衡网络里面最低一个节点做为最新区块，不管怎么去调度都不会导致区块 not found
- 负载均衡网络里面节点块高差是需要监控的，一旦偏差过大需要报警



1.2 使用第三方节点

一般购买多家服务商的节点 alchemy, ankr, getblock, google 和 infura

- 节点活性问题：每条链节点出块时间基本是确定，超过一定时间不出块这个节点认为是有问题的，可以负载均衡网络踢出这个节点，等正常了在纳入进来
- 最新区块：以负载均衡网络里面最低一个节点做为最新区块，不管怎么去调度都不会导致区块 not found
- 负载均衡网络里面节点块高差是需要监控的，一旦偏差过大需要报警



在 FishCake，实际负载是 alchemy 和 ankr

2. 链上交易失败的问题

- 失败的交易重发，或者使用补偿任务单独处理

八.总结

大家可以启动项目，测试钱包的空投，钱包创建之后，提交钱包会获得 FCC 空投，第一次和活动合约进行交互，可以获得空投。

可以去感受一个 BoosterNFT 和 Staking 相关的功能，要深入一下，尤其是需要使用类似项目进行包装的小伙伴。